



Sprint 7 :

Drupal Development Guide

Réalisé par :

Sara Akharraz

Encadrants :

M.Abdelmajid Bendrif

M.Hamza Bahlaouane

Entreprise :

Void

Objectif :

- Se familiariser avec les bases du développement de modules dans Drupal.
- Apprendre à utiliser les routes, contrôleurs, services et l'injection de dépendances.
- Comprendre et utiliser les hooks dans Drupal.
- Découvrir le système de plugins et la création de formulaires.
- Comprendre les différents types de données (Data types) dans Drupal.

Chapitre 1 : Bases du développement de modules :

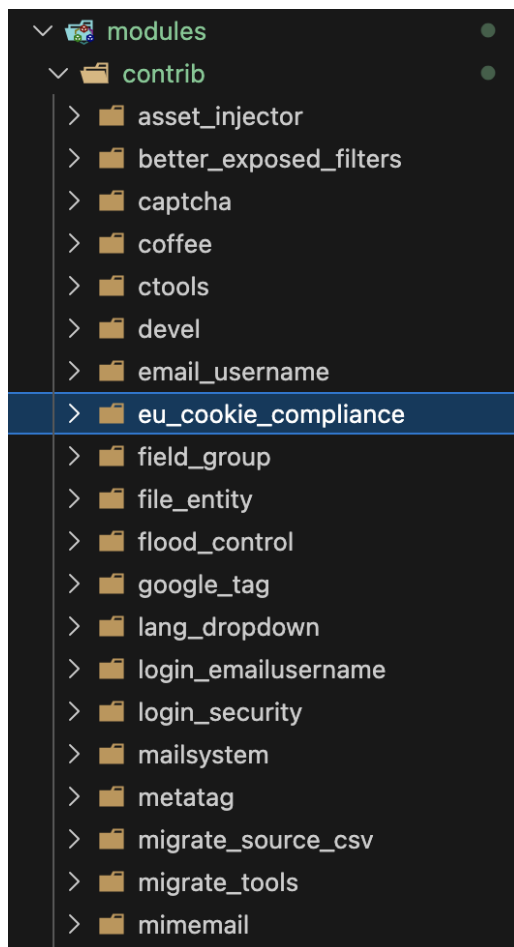
Notes :

Les informations suivantes proviennent de notes prises en regardant la vidéo :

Drupal Development Guide - Preface .

- [Localisation des modules installés](#) :

Les modules installés via Composer dans Drupal se situent dans le répertoire **modules/contrib/** et non dans **vendor/**.



Le dossier vendor contient seulement les **dépendances PHP** installées par Composer.

Les modules en Drupal peuvent provenir de plusieurs sources :

- **contrib** : modules téléchargés et installés
- **core** : modules intégrés à Drupal
- **profil d'installation** : exemple **demo-umami**
- **custom** : modules créés spécifiquement pour un projet

— Le Registre officiel de Drupal :

Drupal possède son propre registre de packages : **packages.drupal.org**. C'est dans ce registre que Composer recherche les modules Drupal.

Comment Composer sait-il où placer les packages téléchargés ?

—> Drupal utilise le package **composer/installers**

Ce package fournit une option extra avec la clé **installer-paths**. Il lit les métadonnées de chaque package et le place au bon endroit selon son type .

Meme si drush est dans drush/Commands/contrib pourquoi en utilisant drush on doit saisir ./vendor/bin/drush ?

Réponse :

Composer crée automatiquement des scripts exécutables dans le répertoire **vendor/bin/** qui pointent vers les commandes principales du package. C'est là que Composer place les points d'entrée du programme.

Le dossier **:drush/Commands** contient seulement les commandes Drupal personnalisées, pas l'exécutable.

—> La commande suivante installe des modules d'exemple avec leurs configurations (pour apprendre le développement Drupal) :

```
composer require 'drupal/examples:^4.0'
```

— Fichiers PO et POT :

Les traductions utilisent deux types de fichiers :

- **PO files** : contiennent les traductions réelles
structure :

```
msgid "Hello"  
msgstr "Bonjour"
```

- **POT files** : sont des templates de traduction .
ils servent de base pour créer les fichiers **.po**

Création des modules :

Hello World Module :

Un module `hello_world` a été développé en suivant le tutoriel : *Step by step tutorial: Hello World* .

Le lien vers le depot GitHub : *hello_world_module* .

- Le fichier `.info.yml` :

C'est un fichier de configuration qui décrit un module à Drupal. Il contient les informations nécessaires pour que Drupal puisse **détecter, charger et afficher le module dans l'interface d'administration**.

```
hello_world.info.yml M X
modules > custom > hello_world > hello_world.info.yml
You, 2 days ago | 1 author (You)
1 name: Hello World Module
2 description: Creates a page showing "Hello World".
3 package: Custom
4 type: module
5 version: 1.0
6 core_version_requirement: ^10 | ^11
```

- Le fichier `.links.menu.yml` :

permet à un module d'ajouter un lien dans les menus de l'interface d'administration de Drupal .

```
hello_world.links.menu.yml X
modules > custom > hello_world > hello_world.links.menu.yml
1 hello.admin:
2   title: 'Hello module settings'
3   description: 'example of how to make an admin settings page link'
4   parent: system.admin_config_development
5   route_name: hello.content
6   weight: 100
```

NB :

weight sert à définir l'ordre d'affichage des éléments (un nombre plus petit apparaît plus haut).

- Le fichier `.routing.yml` :

permet de définir les routes d'un module Drupal.

Il associe une URL à **une action spécifique** afin de déterminer quelle page ou quelle fonctionnalité doit être exécutée lorsqu'un utilisateur accède à cette adresse.

```
hello_world.routing.yml X
hello_world.routing.yml
1 hello_world.content:
2   path: '/hello'
3   defaults:
4     _controller: '\Drupal\hello_world\Controller\HelloController::content'
5     _title: 'Hello World'
6   requirements:
7     _permission: 'access content'
```

NB :

access content est une permission de base dans Drupal qui permet aux utilisateurs de voir le contenu du site.

- Le fichier `controller` :

Un controller dans Drupal est une classe PHP qui contient la logique permettant de générer le contenu d'une page.

```
modules > custom > hello_world > src > Controller > HelloController.php
You, 3 minutes ago | 2 authors (You and one other)

1 <?php
2 /**
3  * @file
4  * Contains \Drupal\hello_world\Controller\HelloController.
5  */
6 namespace Drupal\hello_world\Controller;
7 class HelloController
8 {
9     public function content()
10     {
11         return array(
12             '#type' => 'markup',
13             '#markup' => t('Hello, World!'),
14         );
15     }
16 }

Sidney Harrell, 10 years ago • added controller file
```

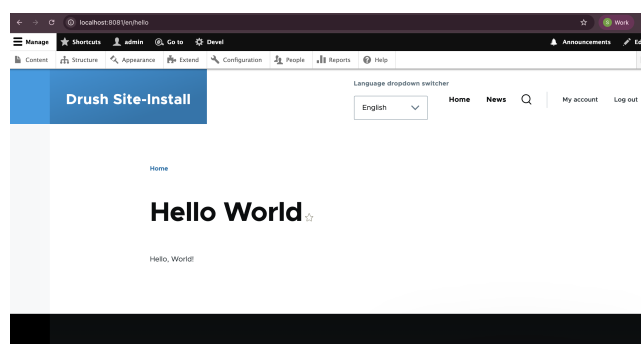
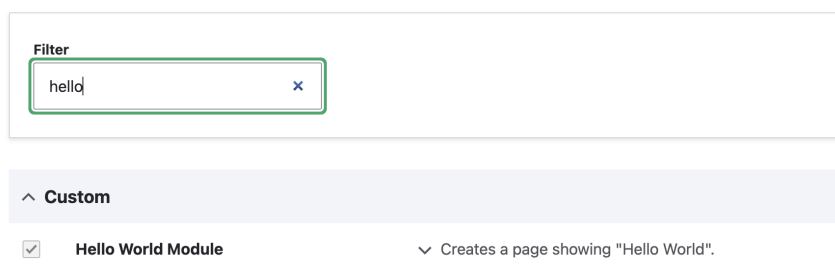
Pour tester le module, j'ai créé un lien symbolique (symlink) dans le projet que j'ai réalisé lors du dernier sprint :

```
> drupal git:(main) x ln -s /Users/void/hello_world_module/modules/custom/hello_world web/modules/custom/hello_world
> drupal git:(main) x ls web/modules/custom/hello_world
web/modules/custom/hello_world
> drupal git:(main) x ls web/modules/custom/hello_world/
LICENSE                hello_world.info.yml      src
README.md              hello_world.links.menu.yml
config                 hello_world.routing.yml
> drupal git:(main) x ./vendor/bin/drush cr
[success] Cache rebuild complete.
> drupal git:(main) x drush pm:list | grep hello
zsh: command not found: drush
> drupal git:(main) x ./vendor/bin/drush pm:list | grep hello
Custom                Hello World Module (hello_world)      Disabled    1
> drupal git:(main) x ' '
```

Pour activer le module :

`./vendor/bin/drush en hello_world -y`

```
> drupal git:(main) x ./vendor/bin/drush en hello_world -y
> [notice] The configuration was successfully updated. 80 configuration objects updated.
> [notice] Message: The configuration was successfully updated. There are /80/ configuration
> objects updated.
>
> [success] Module hello_world has been installed.
> drupal git:(main) x ./vendor/bin/drush pm:list | grep hello
Custom                Hello World Module (hello_world)      Enabled     1
> drupal git:(main) x ' '
```



- Désinstallez les modules puis réinstallez-les. Que se passe-t-il ?

```
drupal git:(main) x ./vendor/bin/drush pm:uninstall hello_world
[success] Successfully uninstalled: hello_world
drupal git:(main) x ./vendor/bin/drush pm:list --type=module | grep hello_world
Custom      Hello World Module (hello_world)  Disabled  1
drupal git:(main) x ./vendor/bin/drush pm:install hello_world
> [notice] The configuration was successfully updated. 87 configuration objects updated.
> [notice] Message: The configuration was successfully updated. There are /87/ configuration
> objects updated.
>
[success] Module hello_world has been installed.
drupal git:(main) x ./vendor/bin/drush pm:list --type=module | grep hello_world
Custom      Hello World Module (hello_world)  Enabled   1
drupal git:(main) x
```

le module se désinstalle correctement et se réinstalle sans conflits.

- Modifiez la valeur `core_version_requirement` vers une version plus basse. Que se passe-t-il ?
mon drupal version :

```
drupal git:(main) x ./vendor/bin/drush status
Drupal version : 11.3.3
```

on diminue la valeur de version dans le fichier `hello_world.info.yml` :

Avant :

```
core_version_requirement: ^10 || ^11
```

Après :

```
core_version_requirement: ^8
```

```
^ Custom
☒ Hello World Module      ^ This version is not compatible with Drupal 11.3.3 and should be replaced.
                               Machine name: hello_world
                               Version: 1
                               Requires: Drupal Core (^8) (incompatible with version 11.3.3)
```

```
drupal git:(main) x ./vendor/bin/drush pm:uninstall hello_world
[success] Successfully uninstalled: hello_world
drupal git:(main) x ./vendor/bin/drush pm:install hello_world

In ModuleInstaller.php line 142:

  Unable to install modules: module 'hello_world' is incompatible with this version of Drupal core.

drupal git:(main) x
```

- Ajoutez le module `workflows` comme dépendance dans vos dépendances. Que se passe-t-il ?

```
web > modules > custom > hello_world > vim hello_world.info.yml
1  name: Hello World Module
2  description: Creates a page showing "Hello World".
3  package: Custom
4  type: module
5  version: 1.0
6  core_version_requirement: ^10 | ^11
7  dependencies:
8  | - drupal:workflows
9
```

```
drupal git:(main) * ./vendor/bin/drush pm:install hello_world
The following module(s) will be installed: hello_world, workflows

Do you want to continue? 
Yes

> [notice] The configuration was successfully updated. 87 configuration objects updated.
> [notice] Message: The configuration was successfully updated. There are /87/ configuration
> objects updated.
>
[success] Module hello_world has been installed.
[success] Module workflows has been installed. (Permissions - Configure)
drupal git:(main) *
```

Drupal installe bien la dépendance workflows .

C'est **une dépendance d'activation** -> mon module hello_world ne s'active que si workflows est activé , si installé et non activé drupal l'active avant d'activer le module .

```
drupal git:(main) * ./vendor/bin/drush pm:list | grep workflows
Core                               Workflows (workflows)           Enabled    11.3.3
drupal git:(main) *
```

NB :

Le fichier .info.yml gère uniquement les dépendances entre modules Drupal et leur activation.

Il ne gère pas les dépendances PHP externes, qui sont prises en charge par composer.json via Composer.

— [Ajoutez un fichier composer.json à vos modules.](#) Que se passe-t-il après l'ajout ?

L'ajout d'un Composer.json n'a pas d'effet immédiat dans Drupal.

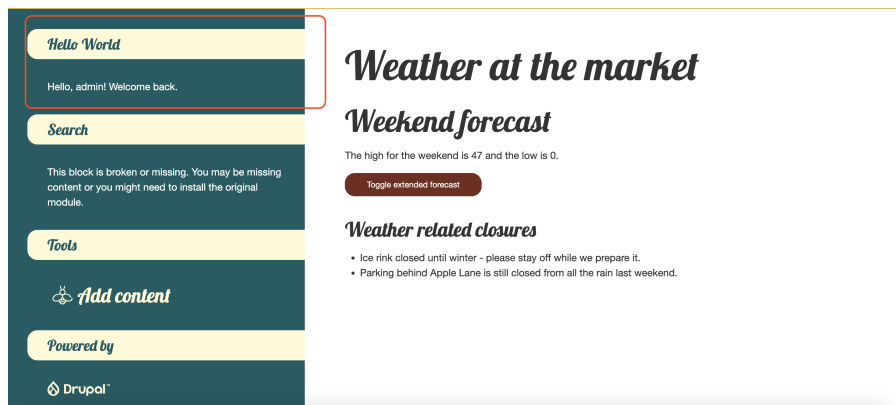
Il permet d'installer le code (modules et librairies) dans le projet et gère les dépendances PHP.

```
composer.json
modules > custom > hello_world > composer.json > ...
You, 17 hours ago | 1 author (You)
1  {
2      "name": "drupal/hello_wolrd_module",
3      "description": "A custom module for learning purpose",
4      "type": "drupal-module",
5      "license": "GPL-2.0-or-later",
6      "autoload": {
7          "psr-4": {
8              "Drupal\\HelloWolrdModule\\": "src/"
9          }
10     },
11     "authors": [
12         {
13             "name": "sara akharraz",
14             "email": "s.akharraz@void.fr"
15         }
16     ],
17     "require": {}
18 }
19
```

Anytown Farmer module :

Le module est développé en suivant les étapes présentées dans le lien suivant : *Concept: Anatomy of a Module* .

Le lien vers le depot GitHub : *Anytown__module* .



Chapitre 2 : Routes, contrôleurs, services et injection de dépendances :

Une **route** dans Drupal définit quelle action doit être exécutée lorsqu'un utilisateur accède à une URL. Elle contient généralement trois parties principales :

- **Path (chemin)** : Définit l'URL qui déclenche l'exécution du code.
- **Defaults (valeurs par défaut)** : Indique l'action à effectuer lorsque la route est appelée.
 - **__controller** : méthode PHP qui génère la page
 - **__form** : classe utilisée pour afficher un formulaire
 - **__title** : titre de la page
- **Requirements (conditions)** : Définit les conditions d'accès à la route, par exemple les permissions nécessaires.

```
anytown.routing.yml M X
web > modules > custom > anytown > anytown.routing.yml
You, 5 hours ago | 1 author (You)
1 anytown.weather_page:
2   path: '/weather/{style}'
3   defaults:
4     _title: 'Weather at the market'
5     _controller: '\Drupal\anytown\Controller\WeatherPage::build'
6     style: 'short'
7   requirements:
8     _permission: 'view weekly weather'
9
10 anytown.settings:
11   path: '/admin/config/system/anytown'
12   defaults:
13     _title: 'Anytown Settings'
14     _form: '\Drupal\anytown\Form\SettingsForm'
15   requirements:
16     _permission: 'administer site configuration'
You, 5 hours ago • Uncommitted changes
```

Types de liens dans Drupal :

Drupal utilise plusieurs types de liens pour organiser la navigation et les actions dans l'interface.

- **Menu items (liens de menu)** :

Ce sont les liens utilisés pour la navigation principale du site, par exemple dans le menu principal ou dans les menus d'administration.



— [Action links \(liens d'action\)](#) :

Ce sont des liens qui permettent d'exécuter une action principale dans un contexte donné, comme ajouter un nouveau contenu ou créer un nouvel élément.



— [Local tasks \(onglets / tabs\)](#)

Ce sont les liens qui apparaissent sous forme d'onglets et permettent de naviguer entre différentes vues d'une même page ou d'une même entité (par exemple View, Edit, Delete).



Permissions dans Drupal :

— **Permissions statiques** Les permissions statiques sont définies manuellement dans un module, généralement dans un fichier : `mymodule.permissions.yml`

Ces permissions sont fixes et ne changent pas selon le contexte.

— **Permissions dynamiques** Les permissions dynamiques sont générées automatiquement par Drupal.

Elles peuvent varier selon le type de contenu ou la configuration du site.

- Créer un article (permission de création)
- Modifier un article (permission de modification)

On peut créer notre permissions dans le fichier **anytown.permissions.yml** :

```
anytown.permissions.yml X
web > modules > custom > anytown > anytown.permissions.yml
You, 2 days ago | 1 author (You)
1 view weekly weather:
2   title: 'View weekly weather'
3   description: 'Allows the user to view the weekly weather at the market.'
```

Permission	Anonymous user	Authenticated user	Content editor	Administrator	Vendor
Anytown					
View weekly weather Allows the user to view the weekly weather at the market.	☐	☐	☐	☒	☐

Comment les permissions sont vérifiées par Drupal ?

Il existe plusieurs façons de vérifier les permissions dans Drupal.

Voici les trois méthodes les plus courantes.

— **Vérification dans une route** : La permission peut être définie directement dans un fichier **.routing.yml**.

Dans ce cas, Drupal vérifie automatiquement l'accès à la page. Si l'utilisateur ne possède pas la permission requise, Drupal bloque l'accès et affiche une erreur 403 (Access denied).

— **Vérification avec `AccessResult`** :

Cette méthode permet de définir des règles d'accès plus dynamiques.

Par exemple, un utilisateur peut modifier un livre : s'il est administrateur ou s'il est le propriétaire du livre. Dans ce cas, on utilise la classe `AccessResult` pour déterminer si l'accès est autorisé ou non.

— **Vérification dans le code avec `hasPermission`** : La méthode `hasPermission()` permet de vérifier une permission directement dans le code.

Contrairement à la vérification dans une route, cette méthode ne bloque pas l'accès à la page, mais permet plutôt de contrôler l'affichage de certains éléments (par exemple afficher ou cacher un bouton ou un lien).

Injection de dépendances et Service Container

Drupal utilise un **Service Container** pour créer et fournir les objets nécessaires aux différentes classes du système. Ce container agit comme une **factory** capable d'instancier les services et d'injecter automatiquement leurs dépendances.

Injection par constructeur

Lorsque la classe implémente `ContainerInjectionInterface`, Drupal appelle automatiquement la méthode `create()` pour récupérer les services depuis le container.

```
public static function create(ContainerInterface $container) {  
    return new static(  
        $container->get('logger.factory')  
    );  
}
```

NB :

La méthode `create()` est utilisée lorsque :

- La classe n'est pas déclarée dans `services.yml`
- Drupal doit instancier la classe via le container

Elle est principalement utilisée pour :

- Controllers
- Forms
- Plugins

Elle n'est généralement pas utilisée pour les services classiques.

Comment découvrir les services existants ?

Pour connaître les services disponibles dans Drupal, on peut consulter la documentation sur `.api.drupal.org`. Cependant, cette documentation ne liste pas les services provenant des modules personnalisés ou contribués.

Une autre solution consiste à installer le module **Devel** (uniquement en environnement local).

```
composer require --dev drupal/devel  
drush en devel
```

Pour afficher tous les services disponibles :

```
drush devel:services
```

Injection sans autowiring

Sans autowiring, les services doivent être récupérés manuellement depuis le container via la méthode `create()`.

```

public function __construct(ClientInterface $http_client) {
    $this->httpClient = $http_client;
}

public static function create(ContainerInterface $container) {
    return new static(
        $container->get('http_client')
    );
}

```

La méthode `create()` permet donc de récupérer explicitement les services depuis le container et de les passer au constructeur.

Injection avec autowiring

L'autowiring permet à Drupal d'injecter automatiquement les services en se basant sur les types définis dans le constructeur, sans écrire la méthode `create()`.

```

use Drupal\Core\DependencyInjection\AutowireTrait;

class WeatherPage extends ControllerBase {

    use AutowireTrait;

    public function __construct(ClientInterface $http_client) {
        $this->httpClient = $http_client;
    }

}

```

En résumé :

- **create()** : injection manuelle des services
- **Autowiring** : injection automatique basée sur les types

Theme hooks et rendu dans Drupal

Un **theme hook** est un identifiant unique qui indique à Drupal comment afficher certaines données. Il permet de faire le lien entre :

- les données PHP provenant d'un module
- le template Twig qui définit l'affichage HTML

Render arrays

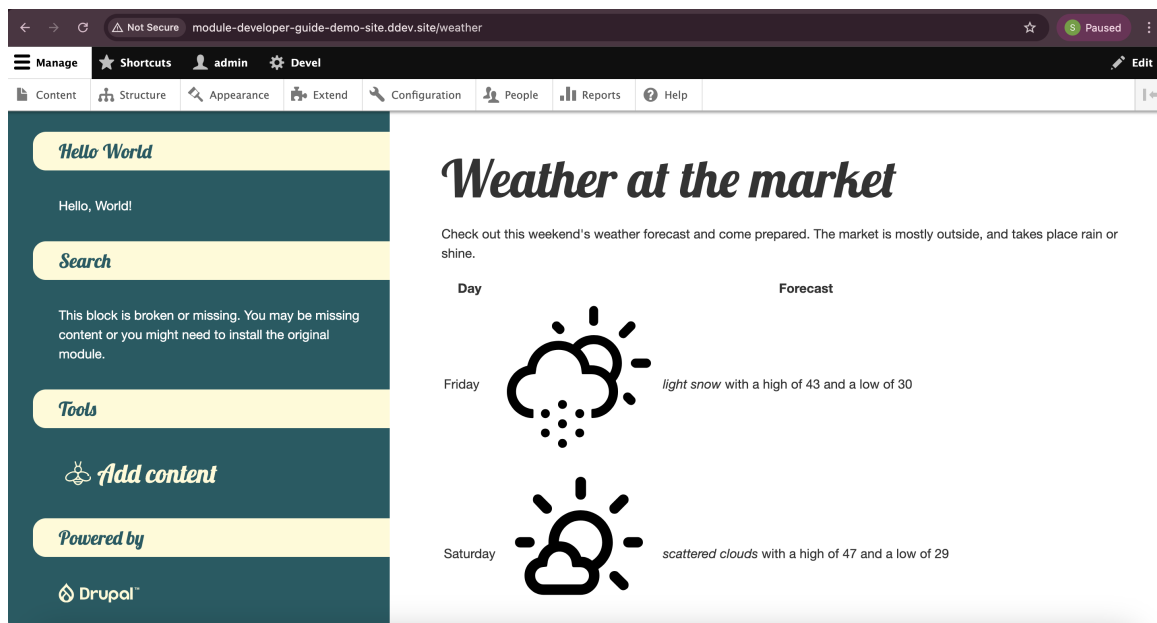
Dans Drupal, les modules doivent retourner du contenu sous forme de **render arrays** plutôt que des chaînes HTML.

Les render arrays sont des tableaux associatifs propres à Drupal qui contiennent :

- les données à afficher
- des informations sur la manière dont ces données doivent être rendues



Ces tableaux permettent à Drupal de contrôler le rendu, la mise en cache et l'affichage du contenu. Après l'ajout des render arrays dans le module anytown :

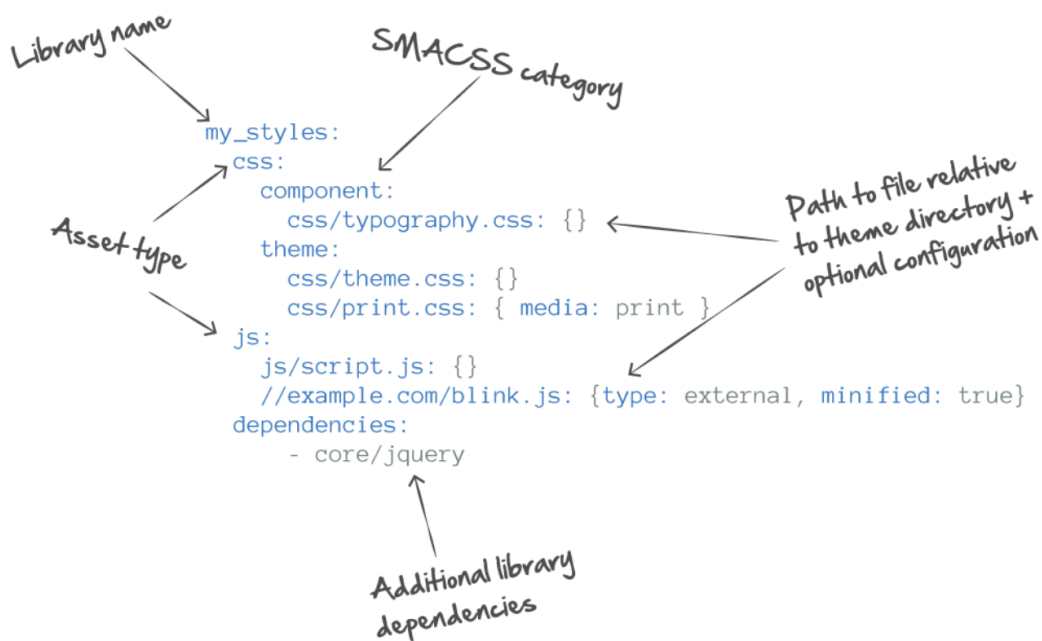


Asset Libraries dans Drupal

Dans Drupal, les **Asset Libraries** permettent de regrouper les fichiers **CSS** et **JavaScript**. Ces bibliothèques sont déclarées dans un fichier nommé :

```
MODULE_NAME.libraries.yml
```

L'idée principale est d'organiser les ressources du site et de ne charger les fichiers CSS ou JavaScript que lorsque cela est nécessaire.



Organisation du CSS avec SMACSS

Les fichiers CSS dans Drupal sont organisés selon la méthodologie **SMACSS** (Scalar and Modular Architecture for CSS). Cette approche permet de classer les styles afin de maintenir un ordre cohérent lors du chargement.

Cette classification s'applique uniquement aux fichiers **CSS** et non au JavaScript.

—> Les principaux groupes SMACSS sont :

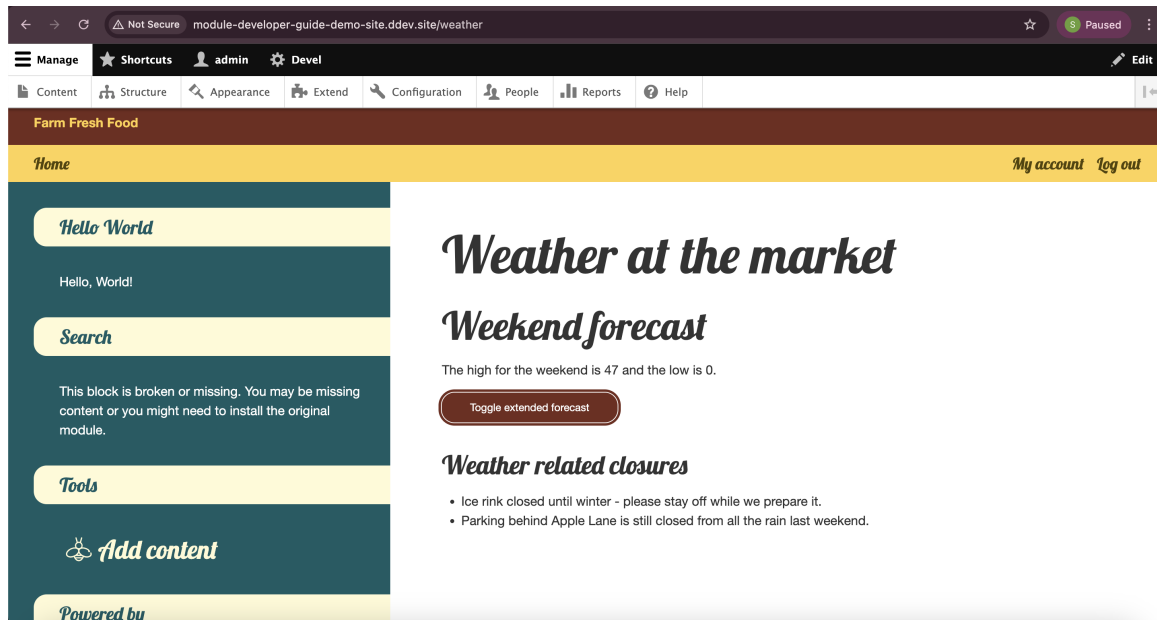
- **Base** : styles de base appliqués directement aux éléments HTML (ex : body, h1, p).
- **Layout** : styles qui définissent la structure de la page (ex : header, footer, sidebar).
- **Component** : éléments d'interface réutilisables (ex : boutons, menus, cartes).
- **State** : styles représentant un changement d'état (ex : actif, caché).
- **Theme** : styles liés à l'apparence visuelle comme les couleurs ou la typographie.

Rôle des thèmes

Dans Drupal, les thèmes peuvent :

- modifier les fichiers CSS ou JavaScript des modules
- ajouter leurs propres styles
- contrôler l'apparence finale du site

Après l'ajout des assets dans le module anytown :



Internationalisation et localisation

Dans Drupal, on distingue deux concepts importants :

- **Internationalisation (i18n)** : préparation du logiciel afin qu'il puisse être traduit. Cela consiste à écrire le code de manière à ce que les textes soient traduisibles.
- **Localisation (l10n)** : traduction réelle du site dans une langue spécifique.

Gestion du pluriel

Certaines phrases changent selon le nombre. Par exemple :

- 1 comment
- 2 comments

Pour gérer correctement ces cas et permettre la traduction, Drupal fournit la fonction `formatPlural()`.

```
$temperature = 2;
```

```
$message = $this->formatPlural(
    $temperature,
    '1 degree',
    '@count degrees'
);
```

Cette fonction choisit automatiquement la forme correcte (singulier ou pluriel) en fonction du nombre.

Placeholders

Les **placeholders** permettent d'insérer des valeurs dynamiques dans un texte traduit.

Les principaux types sont :

- **@variable** : affiche du texte simple en échappant le HTML (sécurité contre les attaques XSS).
- **%variable** : échappe le texte et l'affiche généralement avec un style particulier.
- **:variable** : utilisé pour les URLs ou certains attributs HTML.

StringTranslationTrait

`StringTranslationTrait` est un trait PHP utilisé dans Drupal pour permettre aux classes d'accéder facilement aux fonctions de traduction comme `t()` ou `formatPlural()`.

Il facilite donc l'utilisation du système de traduction dans les classes Drupal.

— Comment puis-je contrôler ou trier les menus (weight) ?

Les menus peuvent être ordonnés avec la propriété `weight`. Un nombre plus petit apparaît plus haut dans le menu.

```
anytown.links.menu.yml ×
web > modules > custom > anytown > anytown.links.menu.yml
You, 4 hours ago | 1 author (You)
1 anytown.weather:
2   title: Weather
3   description: Check this weeks weather
4   route_name: anytown.weather_page
5   menu_name: main
6
7 anytown.settings:
8   title: Anytown Settings
9   parent: system.admin_config_system
10  ✦ route_name: anytown.settings
11  weight: 10 You, 4 hours ago • Hooks & Plugin & Forms
```

— Comment configurer des menus enfants ?

On utilise la propriété `parent` dans le fichier `.links.menu.yml`.

```
anytown.links.menu.yml ×
web > modules > custom > anytown > anytown.links.menu.yml
You, 4 hours ago | 1 author (You)
1 anytown.weather:
2   title: Weather
3   description: Check this weeks weather
4   route_name: anytown.weather_page
5   menu_name: main
6
7 anytown.settings:
8  ✦ title: Anytown Settings
9   parent: system.admin_config_system You, 4 hours ago • Hooks
10  route_name: anytown.settings
11  weight: 10
```

— Comment récupérer un paramètre de requête (query string) dans un contrôleur ?

On utilise l'objet `Request`.

```
$city = \Drupal::request()->query->get('city');
```

URL exemple :

```
/weather?city=Paris
```

— Qu'est-ce que Guzzle et Logger ?

- **Guzzle** : bibliothèque PHP utilisée pour faire des requêtes HTTP (API externes).
- **Logger** : service utilisé pour écrire des messages dans les logs Drupal.

— Comment utiliser Logger pour enregistrer un message dans votre contrôleur ?

```
\Drupal::logger('anytown')->notice('Weather page accessed');
```

Les messages apparaissent dans les logs Drupal :

```
/admin/reports/dblog
```

ou avec la commande :

```
drush ws
```

— Où puis-je trouver les services définis par le core de Drupal ?

Les services du core sont définis dans le fichier :

```
core/core.services.yml
```

— Comment injecter d'autres services dans votre service ?

On utilise l'injection de dépendances via le constructeur.

```
public function __construct(LoggerChannelFactoryInterface $logger) {  
    $this->logger = $logger;  
}
```

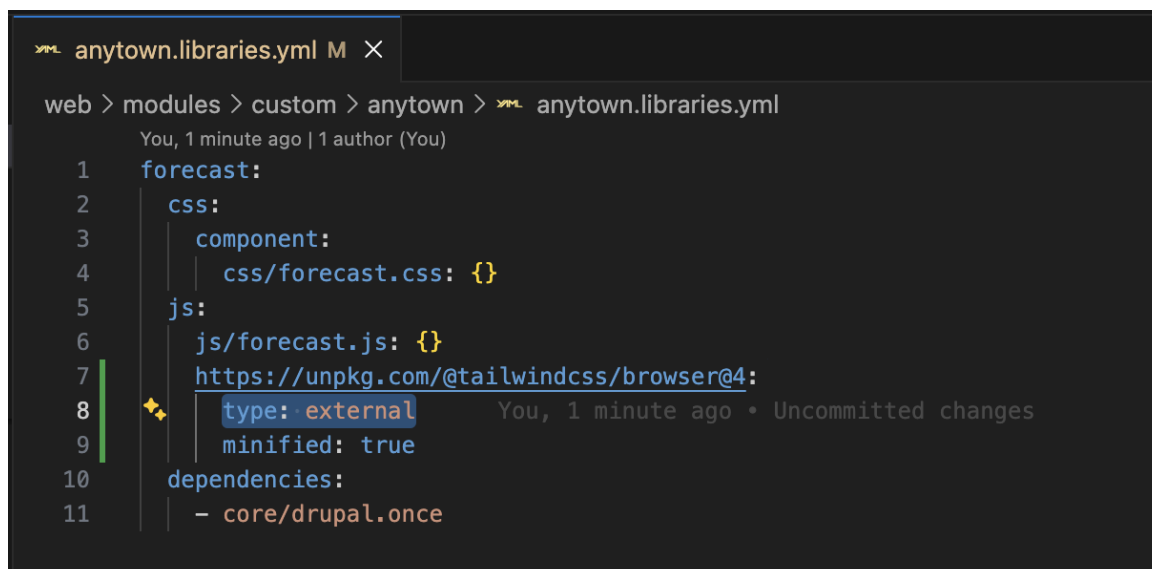
— Comment retourner un template Twig dans un contrôleur ?

On retourne un render array avec le theme hook.

```
return [  
    '#theme' => 'weather_page',  
];
```

— Comment ajouter le JS externe Tailwind ?

On crée une librairie dans `mymodule.libraries.yml`.



```
anytown.libraries.yml M X  
  
web > modules > custom > anytown > anytown.libraries.yml  
You, 1 minute ago | 1 author (You)  
1 forecast:  
2   css:  
3     component:  
4       css/forecast.css: {}  
5   js:  
6     js/forecast.js: {}  
7     https://unpkg.com/@tailwindcss/browser@4:  
8     type: external You, 1 minute ago • Uncommitted changes  
9     minified: true  
10  dependencies:  
11  - core/drupal.once
```


Puis on l'attache dans le contrôleur :

```
'#attached' => [
    'library' => ['anytown/forecast'],
],
```

— Quel est le mot-clé de recherche pour les traductions ?

Pour la chaîne :

```
$this->t('Hello, @name!')
```

Le mot clé recherché est :

```
Hello, @name!
```

— Comment rendre une chaîne traduisible en JavaScript dans Drupal ?

On utilise la fonction :

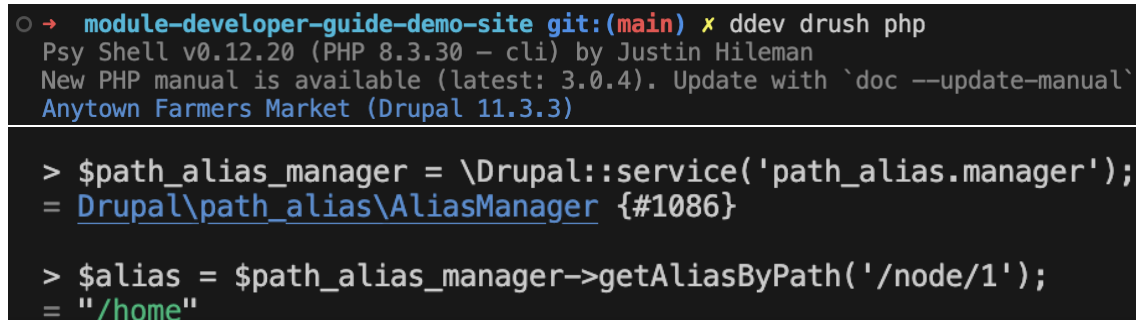
```
Drupal.t('Hello world');
```

— Comment utiliser drush php pour obtenir le service path_alias.manager ?

```
drush php
```

Puis :

```
\Drupal::service('path_alias.manager');
```



```
○ → module-developer-guide-demo-site git:(main) x ddev drush php
Psy Shell v0.12.20 (PHP 8.3.30 - cli) by Justin Hileman
New PHP manual is available (latest: 3.0.4). Update with `doc --update-manual`
Anytown Farmers Market (Drupal 11.3.3)

> $path_alias_manager = \Drupal::service('path_alias.manager');
= Drupal\path_alias\AliasManager {#1086}

> $alias = $path_alias_manager->getAliasByPath('/node/1');
= "/home"
```

— Comment récupérer l'alias d'un node ?

```
$alias = \Drupal::service('path_alias.manager')
->getAliasByPath('/node/1');
```

— Comment utiliser Link et Url pour obtenir l'URL complète ?

```
use Drupal\Core\Url;
use Drupal\Core\Link;

$url = Url::fromRoute('anytown.weather');
$link = Link::fromTextAndUrl('Weather page', $url);
```

— Comment envoyer une réponse JSON dans un contrôleur ?

On utilise JsonResponse.

```
use Symfony\Component\HttpFoundation\JsonResponse;

return new JsonResponse([
    'temperature' => 20
]);
```

Chapitre 3 : Hooks et Events

Dans Drupal, les **hooks** permettent à un module de modifier ou d'étendre le comportement du système sans modifier le code du core.

Ils respectent le principe **Open/Closed** : le système est fermé à la modification mais ouvert à l'**extension**.

Les hooks sont appelés par Drupal à des moments précis pendant l'exécution.

Types de hooks

Il existe plusieurs types de hooks selon leur rôle.

Informational hooks

Ils permettent à un module de fournir des informations à Drupal.

```
function anytown_permission() {
  return [
    'view weather' => [
      'title' => 'View weather forecast',
    ],
  ];
}
```

Dans cet exemple, le module ajoute une nouvelle permission.

Alter hooks

Les alter hooks permettent de modifier des données existantes.

```
function anytown_form_alter(&$form, $form_state, $form_id) {
  if ($form_id == 'user_login_form') {
    $form['name']['#title'] = 'Username or Email';
  }
}
```

Ici, le module modifie un formulaire existant.

Action hooks

Ces hooks permettent de réagir à un événement qui se produit dans Drupal.

```
function anytown_entity_insert($entity) {
  \Drupal::logger('anytown')->notice('New content created');
}
```

Dans cet exemple, le module écrit un message dans les logs lorsqu'un contenu est créé.

Comment découvrir les hooks disponibles ?

La documentation des hooks se trouve principalement dans les fichiers `*.api.php` des modules.

Pour découvrir les hooks disponibles, on peut utiliser la commande :

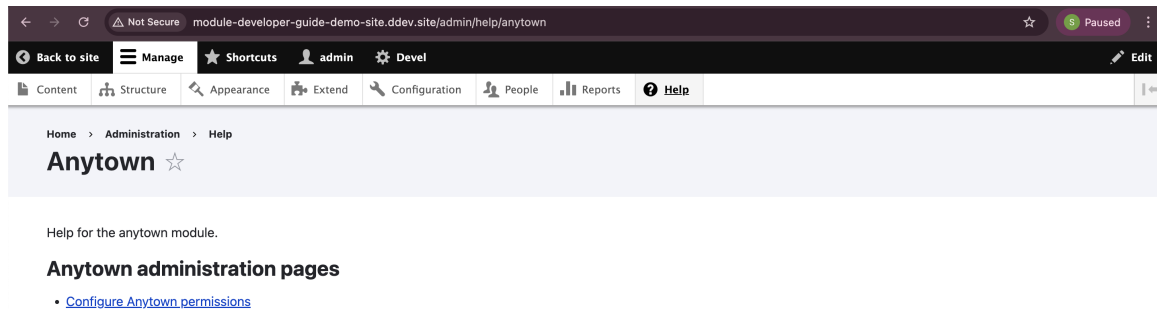
```
ddev drush devel:hook
```

Après avoir ajouter le `hook_help()` dans le module anytown :

Module overviews

Module overviews are provided by modules. Overviews available for your installed modules:

- [Anytown](#)
- [Automated Cron](#)
- [BigPipe](#)
- [Block](#)
- [Block Content](#)
- [Breakpoint](#)
- [CKEditor 5](#)
- [Comment](#)
- [Configuration Manager](#)
- [Configuration Translation](#)
- [Contact](#)
- [Content Translation](#)
- [Contextual Links](#)
- [Custom Menu Links](#)
- [Database Logging](#)
- [Datetime](#)
- [Devel](#)
- [Field](#)
- [Field UI](#)
- [File](#)
- [Filter](#)
- [Help](#)
- [Image](#)
- [Interface Translation](#)
- [Internal Dynamic Page Cache](#)
- [Internal Page Cache](#)
- [Language](#)
- [Link](#)
- [Menu UI](#)
- [MySQL](#)
- [Node](#)
- [Options](#)
- [Password Compatibility](#)
- [Path](#)
- [Shortcut](#)
- [System](#)
- [Taxonomy](#)
- [Text](#)
- [Text Editor](#)
- [Toolbar](#)
- [Update Status](#)
- [User](#)
- [Views](#)
- [Views UI](#)



Events

Les **events** permettent également aux modules de réagir à ce qui se passe dans Drupal.

Un **event** représente une action ou un changement dans le système auquel les modules peuvent s'abonner et réagir.

Le système qui déclenche ces événements est appelé **Event Dispatcher**.

- Quels sont les hooks fournis par le module Metatag et dans quel fichier les trouve-t-on ?

Les hooks du module Metatag sont documentés dans le fichier :

```
metatag.api.php
```

Ce fichier se trouve dans :

```
modules/contrib/metatag/metatag.api.php
```

Les fichiers *.api.php contiennent la documentation des hooks fournis par un module.

- Quel hook est responsable de la modification d'un formulaire ?

Le hook utilisé pour modifier un formulaire est :

```
hook_form_alter()
```

Exemple :

```
function mymodule_form_alter(&$form, FormStateInterface $form_state, $form_id) {  
  
}
```

- Quels sont les arguments reçus par ce hook ?

Le hook reçoit trois arguments :

- **\$form** : la structure du formulaire
- **\$form_state** : l'état du formulaire
- **\$form_id** : l'identifiant du formulaire

- Est-il possible d'étendre un formulaire en ajoutant un simple champ texte ?

Oui, par exemple :

```
function mymodule_form_alter(&$form, FormStateInterface $form_state, $form_id) {

  if ($form_id == 'user_login_form') {

    $form['custom_text'] = [
      '#type' => 'textfield',
      '#title' => t('Custom field'),
    ];

  }

}
```

- Comment utiliser la méthode `isFrontPage` du service `path.matcher` avec un preprocess hook pour ajouter une variable `is_front` dans Twig ?

On utilise le hook :

```
hook_preprocess_page()
```

Exemple :

```
function mymodule_preprocess_page(&$variables) {

  $variables['is_front'] =
    \Drupal::service('path.matcher')->isFrontPage();

}
```

- Comment vérifier que la variable est bien reçue dans Twig ?

Dans un fichier Twig :

```
{{ dump(is_front) }}
```

ou :

```
{% if is_front %}
  <p>Front page</p>
{% else %}
  <p>Not front page</p>
{% endif %}
```

Weather at the market

Weekend forecast

Not front page

The high for the weekend is 47 and the low is 0.

Toggle extended forecast

Weather related closures

- test

— [Comment utiliser hook_page_attachments_alter pour ajouter une balise meta viewport ?](#)

```
function mymodule_page_attachments_alter(array &$attachments) {

  $attachments['#attached']['html_head'][] = [
    [
      '#tag' => 'meta',
      '#attributes' => [
        'name' => 'viewport',
        'content' => 'width=device-width, initial-scale=1, shrink-to-fit=no',
      ],
    ],
    'viewport_meta',
  ];
}
```

```
<meta name="viewport" content="width=device-width, initial-
scale=1, shrink-to-fit=no"> == $0
```

— [Comment utiliser hook_preprocess_menu pour ajouter une classe CSS à tous les éléments du menu ?](#)

```
function mymodule_preprocess_menu(&$variables) {

  foreach ($variables['items'] as &$item) {
    $item['attributes']->addClass('my-custom-class');
  }
}
```

```

▼<li class="my-custom-class menu-item"> == $0
  <a href="/" data-drupal-link-system-path="<font>">Home
  </a>
</li>
▶<li class="my-custom-class menu-item">...</li>
▶<li class="my-custom-class menu-item">...</li>
▶<li class="my-custom-class menu-item">...</li>
▶<li class="my-custom-class menu-item">...</li>

```

- Comment utiliser `hook_preprocess_block` pour modifier le logo du site dans le bloc `system_branding_block` ?

```

function mymodule_preprocess_block(&$variables) {

  if ($variables['plugin_id'] == 'system_branding_block') {

    $variables['site_logo'] =
      'https://static.cdnlogo.com/logos/d/88/drupal-wordmark.svg';

  }

}

```

Chapitre 4 :Plugins et Formulaires dans Drupal

Un **plugin** est un composant réutilisable qui permet d'ajouter ou de modifier une fonctionnalité dans un système existant. Le système de plugins dans Drupal permet d'étendre les fonctionnalités du système de manière modulaire.

Plugin Types

Un **plugin type** est une catégorie de plugins qui définit un type de fonctionnalité dans Drupal.
Par exemple :

- Block plugins
- Field formatters
- Field widgets

Chaque type de plugin correspond à une fonctionnalité spécifique du système.

Plugin Manager

Le **plugin manager** est un service Drupal responsable de la gestion des plugins d'un type donné. Il permet de :

- découvrir les plugins existants
- les enregistrer
- créer leurs instances

Par exemple, le **Block Plugin Manager** :

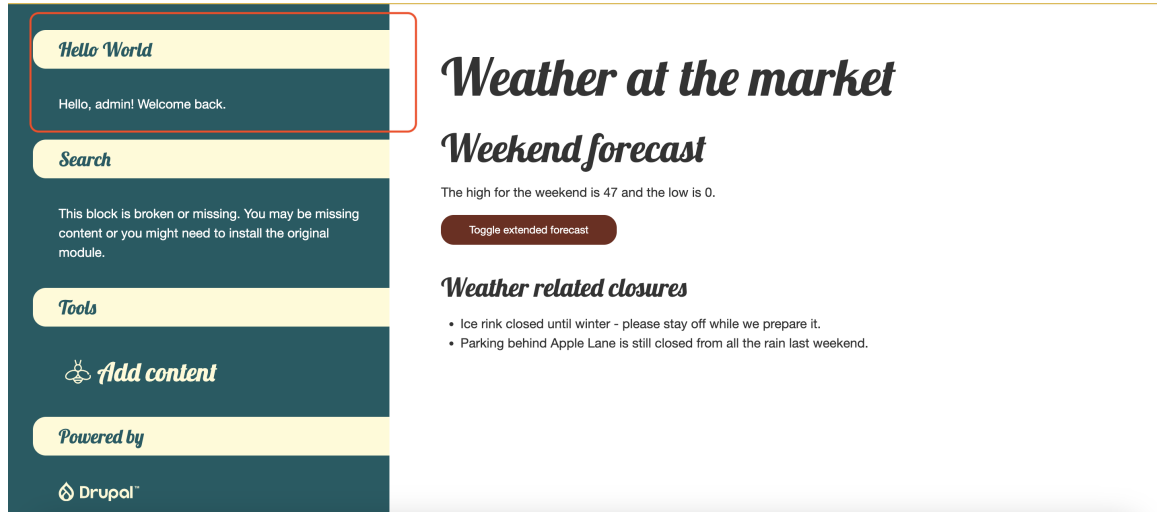
- recherche tous les plugins de type block
- les enregistre

— permet leur utilisation dans Drupal

Plugin Instance

Une **plugin instance** est une implémentation concrète d'un plugin utilisée dans le système. Par exemple, un block spécifique créé par un module est une instance d'un plugin de type block.

Dans la module anytown :



Le formulaire :

A screenshot of the 'Anytown Settings' configuration page in a web browser. The browser address bar shows 'module-developer-guide-demo-site.ddev.site/admin/config/system/anytown'. The page has a dark top navigation bar with links like 'Back to site', 'Manage', 'Shortcuts', 'admin', and 'Devel'. Below this is a light gray breadcrumb trail: 'Home > Administration > Configuration > System'. The main heading is 'Anytown Settings' with a star icon. The settings include a checkbox for 'Display weather forecast', a text field for 'ZIP code of market' containing '90210', and a large text area for 'Weather-related closures'. A 'Save configuration' button is at the bottom.

Error message

Invalid ZIP code

☐ Display weather forecast

ZIP code of market

hello

Used to determine weekend weather forecast.

Weather-related closures

List one closure per line.

Save configuration

- Où peut-on valider les données d'un formulaire dans Drupal?

La validation des données d'un formulaire se fait dans la méthode :

Exemple :

Il est aussi possible d'utiliser un hook :

pour ajouter une validation personnalisée.

On utilise le **FormBuilder** service.

```
$form = \Drupal::formBuilder()->getForm(\Drupal\mymodule\Form\MyForm::class);
```


Cela permet d'afficher le formulaire directement dans le block.

— [Comment rediriger un utilisateur après la soumission d'un formulaire?](#)

On utilise la méthode :

```
$form_state->setRedirect()
```

Exemple :

```
$form_state->setRedirect('mymodule.page');
```

— [Comment afficher un message après la soumission d'un formulaire?](#)

On utilise le service **messenger**.

Exemple :

```
\Drupal::messenger()->addStatus('Form submitted successfully');
```

Les messages apparaissent ensuite dans l'interface utilisateur.

— [Avec #access, comment cacher un champ pour un utilisateur anonyme?](#)

Exemple :

```
$form['secret_field'] = [  
  '#type' => 'textfield',  
  '#title' => 'Secret',  
  '#access' => \Drupal::currentUser()->isAuthenticated(),  
];
```

Le champ sera visible uniquement pour les utilisateurs connectés.

— [Comment regrouper des champs dans un formulaire comme le module field_group?](#)

On peut utiliser un container ou un fieldset.

Exemple avec un fieldset :

```
$form['group'] = [  
  '#type' => 'fieldset',  
  '#title' => 'User information',  
];
```

```
$form['group']['name'] = [  
  '#type' => 'textfield',  
  '#title' => 'Name',  
];
```

```
$form['group']['email'] = [  
  '#type' => 'email',  
  '#title' => 'Email',  
];
```

Cela permet d'organiser les champs dans une section du formulaire.

Chapitre 5 : Data types

Les 4 types d'information dans Drupal :

Drupal gère quatre types principaux de données. Ils se distinguent par leur stockage, leur usage et les API utilisées pour y accéder.

Content

Le **content** correspond aux données visibles par les utilisateurs du site.

Exemples :

- Nodes (articles, pages)
- Users
- Taxonomy terms

Caractéristiques :

- Stocké dans la base de données
- Créé et modifié par les utilisateurs
- Affiché sur le site

API utilisée : **Entity API**

Configuration

La **configuration** définit la structure et le comportement du site.

Exemples :

- Content types
- Fields
- Views
- Site settings

Caractéristiques :

- Partagée entre environnements (dev, staging, production)
- Exportable et déployable

API utilisée : **Configuration API**

Session

La **session** stocke des informations temporaires liées à un utilisateur pendant sa navigation.

Exemples :

- Statut de connexion
- Données temporaires de l'utilisateur

Caractéristiques :

- Spécifique à chaque utilisateur
- Temporaire

Gestion : **Session subsystem (Symfony HttpFoundation)**

State

Le **state** stocke des informations temporaires liées à l'état du site.

Exemples :

- Dernière exécution de cron
- Dernière maintenance du site

Caractéristiques :

- Temporaire
- Non exporté entre environnements

API utilisée : **State API**

Résumé

- **Content** : données du site visibles par les utilisateurs (Entity API).
- **Configuration** : structure et paramètres du site (Configuration API).
- **Session** : données temporaires propres à un utilisateur.
- **State** : données temporaires liées à l'état du site (State API).

The screenshot shows a web browser window with the URL `module-developer-guide-demo-site.ddev.site/node/4/status`. The page has a dark header with navigation links: **Manage**, **Shortcuts**, **admin**, and **Devel**. Below the header is a yellow banner with *Home* and *Sweet Honey* on the left, and *My account* and *Log out* on the right. A row of buttons includes **View**, **Edit**, **Attendance**, **Delete**, **Revisions**, and **Devel**. The main content area features the heading *Weekly status update* and the text "Updating status for vendor: **Sweet Honey**". There is a section titled **Attending** with a checkbox and the instruction "Check this box if you plan to attend this weekends market." Below this are two required text input fields: **Contact name *** (containing "sara") and **Contact email *** (containing "sara@example.com"). At the bottom is a **Update status** button.



— Comment utiliser Drush pour afficher la configuration du module eu_cookie_compliance ?

On peut utiliser la commande :

```
drush php:eval "var_dump(\Drupal::config('eu_cookie_compliance.settings')->get());"
```

Cela permet d'afficher la configuration enregistrée pour ce module.

— Où se trouve le fichier de configuration du module eu_cookie_compliance ?

Les fichiers de configuration se trouvent généralement dans :

```
config/install/
```

Par exemple :

```
eu_cookie_compliance.settings.yml
```

— Pourquoi avons-nous besoin d'un fichier schema.yml ?

Le fichier **schema.yml** permet de définir la structure des configurations d'un module. Il sert à :

- décrire les types de données
- valider les valeurs de configuration
- permettre la traduction des configurations

— Comment charger un node dans Drupal ?

Exemple :

```
$node = \Drupal\node\Entity\Node::load($nid);
```

où \$nid est l'identifiant du node.

— Comment récupérer les valeurs des champs d'un node ?

Exemple :

```
$title = $node->getTitle();
$value = $node->get('field_name')->value;
```

- Comment modifier un champ et sauvegarder le node ?

Exemple :

```
$node->set('field_name', 'New value');  
$node->save();
```

- Qu'est-ce qu'une Constraint ?

Une **constraint** est une règle de validation appliquée à une entité ou à un champ. Elle permet de vérifier que les données respectent certaines conditions avant d'être enregistrées.

- Qu'est-ce qu'un View Builder ?

Un **View Builder** est une classe responsable du rendu d'une entité.

Il utilise les **display modes** pour afficher l'entité de différentes manières, par exemple :

- full
- teaser
- custom display modes

- Dans les Entity Queries, quel est le rôle de accessCheck ?

La méthode :

```
->accessCheck(TRUE)
```

permet de vérifier les permissions de l'utilisateur courant afin de ne retourner que les entités auxquelles il a accès.

- Comment récupérer la traduction française d'un node ?

Exemple :

```
$node_fr = $node->getTranslation('fr');
```

Cela retourne la version traduite du node en français si elle existe.